

Bittorrent Network

Monkeys:

Gabriel West

Yifei.Xie

Hongxiang He



Scenario

- Peer to peer file sharing
- N people want a file, that file is split into parts and distributed to the players.
- These people need to coordinate with other players to finish the download of the file as fast as possible.

Mechanism - Repetition

- Our formulation of the original game was essentially a reskin of the public good game with a standard utility function(does not account for inequity aversion) in the Fehr-Schmidt paper
- In the previous formulation there is one and only one Nash equilibrium and it is not pareto.
- We want to make our game more realistic while also trying to either have a Nash equilibrium that is also pareto or a Nash equilibrium with a net positive social utility.
 - We try do both at the same time by modeling our scenario with a repeated game.

Definition of the Extensive Form Game

- N players
- All N players want to download a file that is split into k parts, [k] will be the set of parts of this file.
- M rounds

$$f : [n] \rightarrow P([k])$$

- Let f be called the part assignment function where,

$$\bigcup_{j \in [n]} f(j) = [k]$$

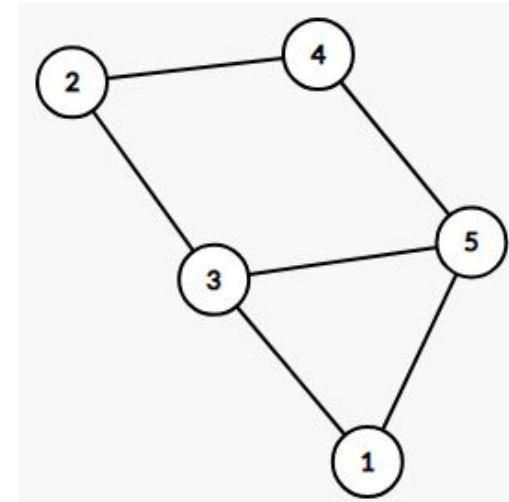
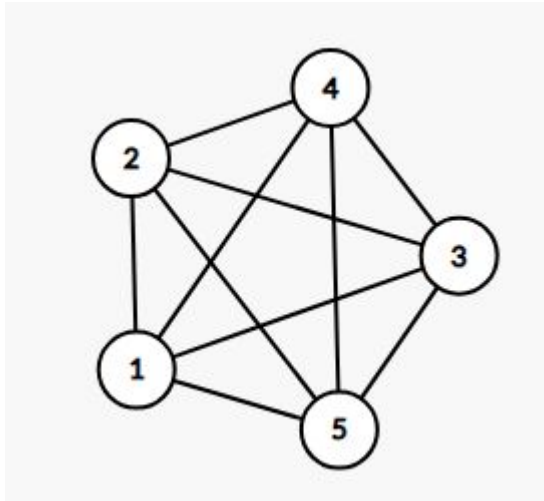
Example of a part assignment function in a game with $n = 5$ and $k = 5$

$$f(1) = \{1,2,3\} \quad f(2) = \{4,5\}$$

$$f(3) = \{1,2\} \quad f(4) = \{\} \quad f(5) = \{1,2,3,4,5\}$$

Definition of the Extensive Form Game cont.

- We define a connected simple graph $G = (V, E)$ where each vertex in V corresponds to a player and $(u, v) \in E$ if and only if players u and v are connected in the game.



- For simplicity we will only be considering games where the network connection graph is a complete graph.

Definition of the Extensive Form Game cont.

- Each player i has a download and upload bandwidth, $x_i \in \mathbb{N}$ and $y_i \in \mathbb{N}$ respectively.
- If player j and player i are adjacent in G then they both know each other's bandwidths and parts.
- Each round of the repeated game has two phases, the request phase and the request handling phase.

Definition of the Extensive Form Game - Request Phase

- Each player simultaneously makes requests to other players for a part p from player j at a rate b_p .
- We assume that players cannot download the same part from multiple players simultaneously.

A strategy for player i is a k element set $R_i = \{r_1, \dots, r_k\}$ where $r_p \in (a, p, b_p)$, $a \in [n] \setminus \{i\}$, $b_p \in [x_i] \cup \{0\}$ and $p \in [k]$.

There is also the condition that the set $\{p : (a, p, b) \in R_i\} = [k]$.

$\sum_{p \in [k]} b_p$ cannot not exceed x_i

- r_p is the request that player i makes to player a for part p and a rate of b_p
- The game is simultaneous at each phase so players do not know what other players requested until the next phase

Definition of the Extensive Form Game - Request Handling Phase

Request phase strategies for player i

A strategy for player i is a k element set $R_i = \{r_1, \dots, r_k\}$ where $r_p \in (a, p, b_p)$, $a \in [n] \setminus \{i\}$, $b_p \in [x_i] \cup \{0\}$ and $p \in [k]$.

There is also the condition that the set $\{p : (a, p, b) \in R_i\} = [k]$.

$\sum_{p \in [k]} b_p$ cannot not exceed x_i

- r_p is the request that player i makes to player a to download part p at a rate of b_p

Request handling phase strategies for player i

Let R_j be the strategy any player $j \neq i$ chose in the most recent request phase, then a strategy for player i is $S_i = \{(j, p, c) : (i, p, b) \in R_j \text{ and } c < b\}$ where $\sum c \leq y_i$.

- In both phases there exists strategies that are not 'meaningfully different'

Definition of the Extensive Form Game - Game Tree

- The game is not complete if the connection graph G is not a complete graph.

Information Partition:

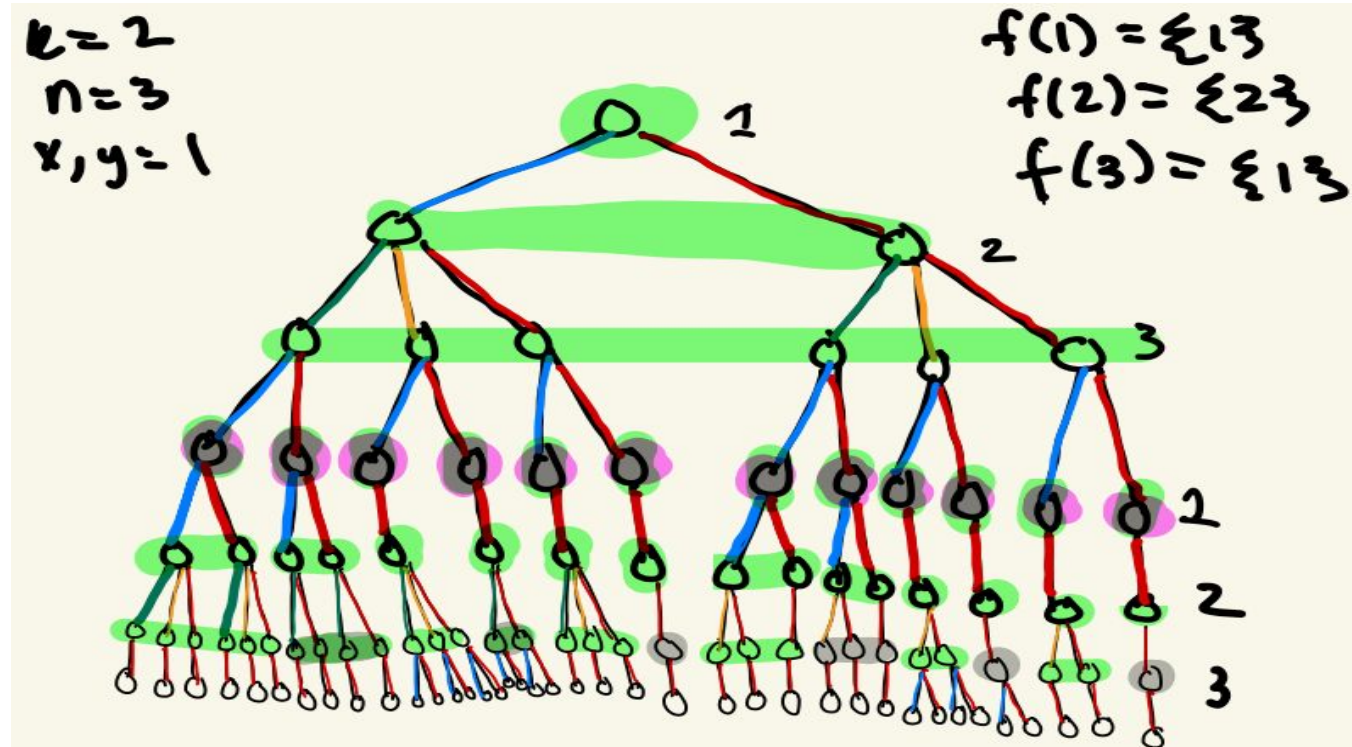
Observation 1: Two non-player 1 nodes are in the same information set if and only if they are in the same level and they share the same most recent player 1 ancestor node. Player 1 always has perfect information.

Green: send or accept request to/from player 1

Blue: send or accept request to/from player 2

Orange: send or accept request to/from player 3

Red: Do nothing.



Definition of the Extensive Form Game - Payoffs

- We want the payoff at the terminal nodes to be related to how long it takes for a player to download the file and if they do not finish we want how much.
- The utility function is instead a cost function analogous to the delay function in the congestion game.
- Each round takes t seconds
- Let w be the size of each part.

Definition round m j-i part-p download sequence: Let m be the round that this sequence will start at. Then consider a sequence of edges in T /sequence of moves $e_1, \dots, e_\beta$ where e_{2b-1} is move $R_{i_{b+m-1}}$ in the tree and e_{2b} is the move $S_{j_{b+m-1}}$ in the tree. This sequence is a **round m j-i part-p download sequence** if $(j, p, z) \in R_{i_{b+m-1}}$ and $z \neq 0$ for all e_{2b-1} and $(i, p, c) \in S_{j_{b+m-1}}$ and $c \neq 0$ for all e_{2b} and $\sum c \geq w$.

A **round m j-i part-p download sequence** is **valid** if and only if $p \in f(j)$ or there is a valid round m' γ -j download sequence that ends before round m

Definition of the Extensive Form Game - Payoffs cont.

- Each round takes t seconds
- Let w be the size of each part

Definition round m j-i part-p download sequence: Let m be the round that this sequence will start at. Then consider a sequence of edges in T /sequence of moves $e_1, \dots, e_\beta$ where e_{2b-1} is move $R_{i_{b+m-1}}$ in the tree and e_{2b} is the move $S_{j_{b+m-1}}$ in the tree. This sequence is a **round m j-i part-p download sequence** if $(j, p, z) \in R_{i_{b+m-1}}$ and $z \neq 0$ for all e_{2b-1} and $(i, p, c) \in S_{j_{b+m-1}}$ and $c \neq 0$ for all e_{2b} and $\sum c \geq w$.

A **round m j-i part-p download sequence** is **valid** if and only if $p \in f(j)$ or there is a valid round m' γ -j download sequence that ends before round m

- Let Q be a path from the root node to a terminal node v , let q be the earliest terminating (with respect to rounds) sub-path such that

$$P_i = \{p \in [k] : p \in f(i) \text{ or there exists a valid round } m \text{ j-i part-p download sequence in } Q\} = [k]$$

- Then let d be the round that q terminates on then the download time for player i is $d \cdot t$, if no such path q exists for player i then they did not finish the file and get a download time of **infinity**.

External form to normal form - strategy profile example.

Green: send or accept request to/from player 1

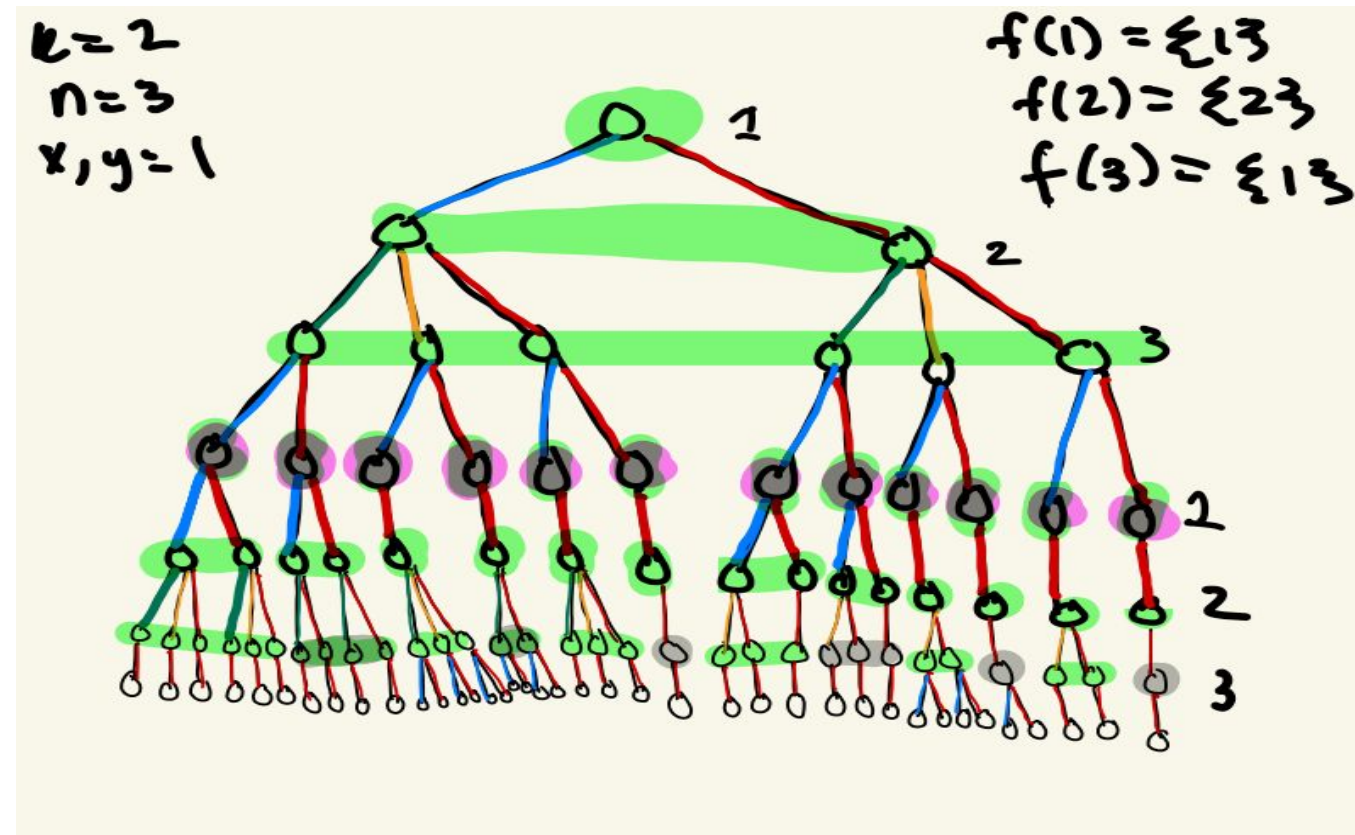
Blue: send or accept request to/from player 2

Orange: send or accept request to/from player 3

Red: Do nothing.

Player 1	<u>B</u> BBRR RRRRR RRR	INF
Player 2	<u>O</u> GROR ORRRR RRR	1
Player 3	<u>B</u> RRBB RRRRR BRR	1

Let $t = 1, R = 1$



External form to Normal Form cont.

Proposition 2: There are $(n-1)^k \binom{k+x_i}{k}$ different strategies that player i can choose from in any request phase.

Proposition 3: There are at most $\binom{(n-1)k}{y_i} \binom{2y_i}{y_i}$ different strategies for player i in the request phase.

The overall objective of each method

single round method

```
run_simulation(5, False)
```

The results from a single run can intuitively demonstrate the behavioral patterns during the simulation process, by animation and all kind of matrix which is is useful for debugging the model or presenting the experimental details.

With the detailed **matrices and even animations** (if enabled), we can see who's doing what, when for how long.

multi-rounds method

```
calculate_strategy_averages (num_simulations=200, num_peers=10)
```

This method performs multiple simulations (200 runs, each with 10 peers) to calculate the average download time for each strategy. The final output provides the average performance metrics for each strategy.

Experimental Goal

1. By aggregating and averaging performance data across multiple runs, the method eliminates the impact of randomness in a single simulation. It provides a more accurate assessment of **how different strategies perform in repeated experiments**.

Performance Comparison:

2. The method outputs the average download time for each strategy, making it easy to compare which strategies perform better in large-scale simulations.

It helps identify significant differences, such as whether "free_rider_strategy" leads to longer download times or whether certain strategies (e.g., "random_bandwidth") exhibit more consistent performance.

The Bittorrent simulator

Scan to view the python code of
The_Best_Bittorrent_Forever



Scan to view the pseudocode



https://github.com/cccpStephen/Collective_action_problem_Monkeys/blob/simulation-python-code/The_best_Bittorrent_Forever.ipynb

What we will talk

The expected output

How we set up the environment

The strategies, how it works

```
run_simulation(10, False)
```

Peer Initial Values:

Peer ID	Upload BW	Download BW	Strategy
0	2.00	32.00	free_rider_strategy
1	3.00	42.00	demand_priority
2	3.00	40.00	random_bandwidth
3	3.00	43.00	free_rider_strategy
4	4.00	45.00	free_rider_strategy
5	1.00	45.00	random_bandwidth
6	1.00	30.00	free_rider_strategy
7	5.00	31.00	demand_priority
8	4.00	46.00	free_rider_strategy
9	4.00	43.00	equal_distribution_strategy

Download Times:

- The output first print out the upload bandwidth (Upload BW), download bandwidth (Download BW), and strategy used by each peer
- Integer

Peer 0: 25.4

Peer 1: 25.6

Time Matrix:

	Piece 0	Piece 1	Piece 2	Piece 3	Piece 4	Piece 5	Piece 6	Piece 7	Piece 8	Piece 9
Peer 0	8.4	—	—	13.8	8.4	—	25.4	12.6	—	21.4
Peer 1	4.6	14.1	8.5	—	10.5	25.6	6.6	—	5.9	14.3
Peer 2	14.3	—	4.8	25.7	—	—	29.7	6.7	4.8	31.7
Peer 3	—	—	—	—	—	—	—	—	8.6	—
Peer 4	—	—	non-finished	14.3	6.7	3.0	8.6	67.9	—	3.0
Peer 5	4.8	—	—	14.4	—	—	25.8	—	—	—
Peer 6	10.6	14.4	4.9	31.0	25.8	6.9	29.7	61.0	14.4	—
Peer 7	4.9	—	14.4	25.8	—	—	—	—	—	8.6
Peer 8	—	—	—	30.5	—	—	—	non-finished	—	—
Peer 9	8.6	—	—	33.6	—	—	—	—	—	—

Total Finished Time for Each Peer:

- Peer 0: 25.4
- Peer 1: 25.6
- Peer 2: 31.7
- Peer 3: 8.6
- Peer 4: non-finished
- Peer 5: 25.8
- Peer 6: 61.0
- Peer 7: 25.8
- Peer 8: non-finished
- Peer 9: 33.6

This section displays the total time it took for each peer to complete all its downloads

If a peer failed to download all requested pieces, it is marked as "non-finished."

- Example: Peer 0 finished all downloads in 25.4 seconds, while Peer 4 had incomplete downloads and is marked as "non-finished."

The Total Finished Time is not always the same as the maximum value in the Time Matrix because:

- Connections between peers are sequential, so simultaneous transferring the different pieces between the same pair are not allowed.

+ Code + Text

```
▶ final_averages = calculate_strategy_averages(num_simulations=300, num_peers=10)

print("Final Average Download Times for Each Strategy:")
for strategy, avg_time in final_averages.items():
    print(f"{strategy}: {avg_time:.2f} units")
```

```
↔ Final Average Download Times for Each Strategy:
random_bandwidth: 7.70 units
tit_for_tat: 8.12 units
equal_distribution_strategy: 9.65 units
demand_priority: 8.24 units
free_rider_strategy: 11.16 units
```

Let's discuss how it is set up

Some default global value
Define TOTAL_PIECES = 10
Define SIMULATION_TIME = 100
The File size

Class Peer:

Constructor(env, id, upload_speed, download_speed, animation, strategy):

Initialize peer properties:

- upload_speed: Peer's upload bandwidth (randomly generated).
- download_speed: Peer's download bandwidth (randomly generated).
- strategy: Assigned strategy for bandwidth allocation.
- file_pieces: Set of file pieces the peer owns. Make sure each peer have one pieces

Method add_peer(peer):

Add another peer to this peer's connection list.

Method request_piece(piece, target):

If target peer owns the requested file piece:

Log the request in request_log.

Wait for the target peer's upload resource.

Transfer data based on the target peer's strategy.(explained later by Yifei)

Update file_pieces and record the download time.

Else:

Log that the piece is unavailable.

Method run():

Continuously check for missing file pieces.

Randomly shuffle peers and request missing pieces sequentially.

Stop when all pieces are downloaded.

Strategy in the request phase

Players decide to request pieces from which players?

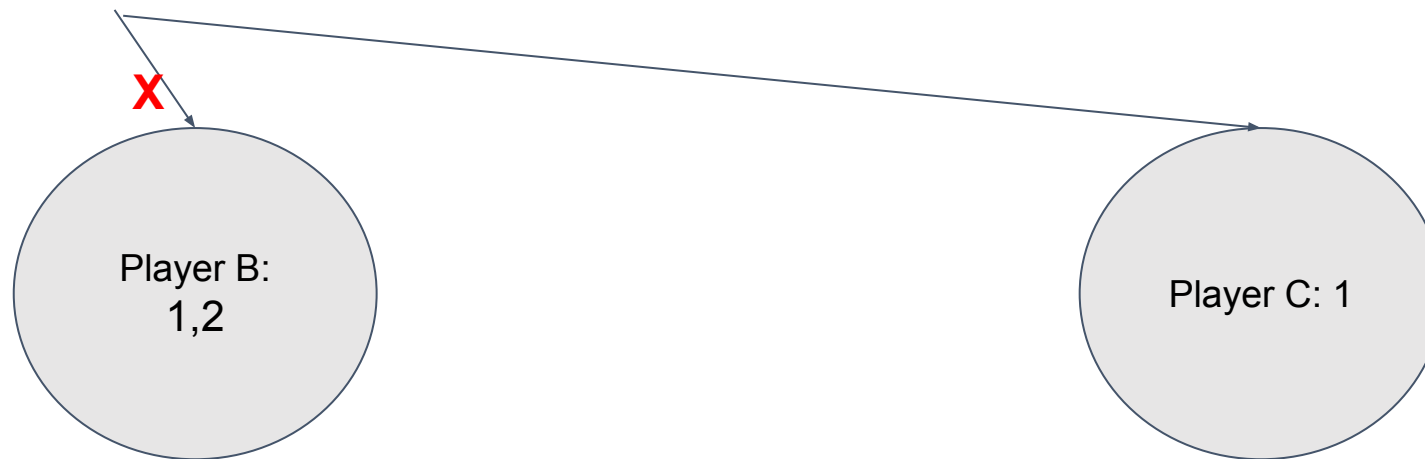
Player A requests piece 1



Strategy in the request phase

Players decide to request pieces from which players?

Player A requests piece 1



If A has already requested another piece from B

Given a situation when A has already requested another piece from B, it is better not to request for 2 pieces from one player at the same time because the new request has to wait until the former piece is executed and the total time of each round may be accumulated. Choosing player C is a better choice.

Five Strategies in the request handling phase

Players decide how much bandwidth they should allocate.

(1) Equal distribution strategy

Evenly distribute upload bandwidth based on the current number of requests, which was counted during the request phase.

(2) Tit for tat strategy

At the very beginning before the first distribution, all players get nothing from others, tit for tat doesn't work.

$$B_{M \rightarrow N} = \frac{U_{M \rightarrow N}}{\sum_i U_{i \rightarrow N}} \cdot U_N$$

N respond to the request from M: This formula represents the amount of data M has uploaded to N, compared to the total data uploaded to N by all peers, then times the upload bandwidth of N.

```
DEFINE tit_for_tat(peer, target_peer_id):  
    - If peer hasn't downloaded anything yet, fall back to equal_distribution_strategy  
    - Otherwise, check how much target peer has downloaded from this peer  
    - Calculate bandwidth ratio based on upload history  
    - RETURN the bandwidth ratio multiplied by the upload speed
```

Five Strategies in the request handling phase

Players decide how much bandwidth they should allocate.

(3) Demand priority strategy

The player with fewer requests will be prioritized.

$$\text{demand_ratio} = \frac{\frac{1}{\text{target_requests}}}{\sum_{i=1}^n \frac{1}{\text{len}(\text{request_list}_i)}}$$

Denominator is the inverse of total normalized demand across all players.

Numerator is the inverse of requests made by the target player.

(4) Free rider strategy

Players don't upload regardless of others' strategies.

(5) Random distribution strategy